Лабораторная работа №1

Основы 3D-графики

Вариант: 1 — Куб
Студент: Акифьев А. С.
Группа: М8О-315БВ-24 **Преподаватель:** Владимир
Дата: 5 октября 2026 г.

---

1. Цель работы

Изучение основ 3D-графики: построение простых 3D-объектов, проекция на 2D-плоскость, работа с матрицами перспективы, ортографической проекции и аффинными преобразованиями.

---

2. Задание

Вариант №1: Куб.

Дополнительные задания (выполнены все, на оценку 5):

1. Переключение матрицы проекции (ортографическая и перспективная) в интерфейсе
2. Элементы интерфейса для изменения позиции, поворота и растяжения фигуры
3. Движение и поворот фигуры по сложной траектории, пауза/воспроизведение анимации, изменение скорости и параметров траектории
4. Изменение цвета фигуры с помощью элементов интерфейса
5. Процедурные цвета вершин, умножение на выбранный цвет
6. Несколько объектов на сцене с отдельными наборами дескрипторов

---

3. Используемые технологии

- **Язык программирования:** C++20
- **Графическое API:** Vulkan 1.1+
- **Библиотеки:**
  - GLFW — создание окна и обработка ввода
  - ImGui — интерактивный интерфейс
  - GLM — линейная алгебра (матрицы, векторы)
  - VMA (Vulkan Memory Allocator) — управление памятью GPU
  - vk-bootstrap — упрощённая инициализация Vulkan
- **Система сборки:** CMake 3.20+
- **Компилятор:** GCC 14.2.0 (MSYS2 UCRT64)

---

4. Реализация

4.1. Геометрия куба

Куб задан 8 вершинами и 36 индексами (12 треугольников).
Каждая вершина содержит:

- позицию (vec3) — координаты в локальной системе от (-0.5, -0.5, -0.5) до (0.5, 0.5, 0.5);
- цвет (vec3), назначенный процедурно в зависимости от координат в локальной системе фигуры.

Индексы задают 12 треугольников (по 2 на каждую из 6 граней), что даёт правильный куб с общим vertex-буфером.

4.2. Графический конвейер Vulkan

Создан графический конвейер со следующими настройками:

- Топология: TRIANGLE_LIST
- Cull mode: NONE — видны все грани, что удобно для демонстрации
- Front face: COUNTER_CLOCKWISE
- Depth test: включён, VK_COMPARE_OP_LESS
- Dynamic state: VIEWPORT и SCISSOR (для корректной работы при изменении размера окна)
- Дескрипторный набор: один uniform-буфер на объект

- Формат вершин: позиция (offset 0) + цвет (offset 12), stride = 24 байта

4.3. Шейдеры

Вершинный шейдер (cube.vert):

- Принимает на вход позицию и цвет вершины;
- Применяет матрицы model, view, proj из uniform-буфера;
- В зависимости от флага use_vertex_color передаёт во фрагментный шейдер либо базовый цвет, либо vertex_color × base_color.

Фрагментный шейдер (cube.frag):

- Выводит полученный цвет как итоговый.

4.4. Матричные преобразования

Для каждой вершины вычисляется:

```
gl_Position = proj × view × model × vec4(position, 1.0)
```

- Model — аффинные преобразования объекта: перевод, поворот по трём осям, масштаб;
- View — вид из камеры, расположенной в точке (0, 2.5, 6), направленной в начало координат;
- Projection — либо glm::perspective(45°, aspect, 0.1, 100), либо glm::ortho(-5×aspect, 5×aspect, -5, 5, 0.1, 100).

Так как в Vulkan ось Y направлена вниз (в отличие от OpenGL), применяется инверсия: proj[1][1] *= -1.0f.

4.5. Синхронизация и рендеринг

- Двойная буферизация через VkSemaphore (доступность изображения и завершение рендеринга);
- Fence для ожидания завершения предыдущего кадра;
- Command buffer перезаписывается каждый кадр: vkBeginCommandBuffer → vkCmdBeginRenderPass → привязка pipeline, vertex/index буферов, дескрипторов → vkCmdDrawIndexed × N → vkCmdEndRenderPass → vkEndCommandBuffer;
- Очистка цвета (0.05, 0.05, 0.10) и глубины (1.0).

4.6. Дополнительные задания — детали реализации

Задание 1. Переключение проекции. Реализовано через чекбокс Perspective Projection в ImGui. При снятии галочки матрица proj меняется на ортографическую. Обе матрицы пересчитываются каждый кадр с учётом текущего aspect ratio окна.

Задание 2. Трансформации фигуры. Слайдеры Position (X/Y/Z), Rotation (X/Y/Z, в градусах), Scale (X/Y/Z) для каждого объекта. Значения напрямую идут в матрицу model через glm::translate, glm::rotate, glm::scale.

Задание 3. Анимация по сложной траектории. Каждый объект движется по окружности радиуса animation_radius с фазовым сдвигом phase = i × 2π / N, где i — номер объекта, N — их количество. Дополнительно объект вращается вокруг оси Y со скоростью, пропорциональной времени. В интерфейсе есть:

- чекбокс Animation Playing (пауза/воспроизведение);
- слайдер Animation Speed (скорость анимации);
- слайдер Animation Radius (радиус траектории);
- кнопка Reset Time (сброс времени).

Задание 4. Цвет фигуры. ColorEdit4 (RGBA) для каждого объекта. Цвет передаётся в uniform-буфер как base_color и умножается на цвет вершины (если включён Use Vertex Color) или используется напрямую.

Задание 5. Процедурные цвета вершин. Цвет каждой вершины вычисляется из её нормализованных координат в локальной системе: color = (x + 0.5, y + 0.5, z + 0.5) то есть каждая вершина куба получает уникальный цвет. При включении Use Vertex Color итоговый цвет = vertex_color × base_color, что даёт наглядную демонстрацию умножения.

Задание 6. Несколько объектов с отдельными дескрипторами. До 5 кубов на сцене. Каждый объект имеет:

- собственный VkBuffer uniform-данных;
- собственный VkDescriptorSet из общего пула.

Все объекты используют общий vertex-буфер и index-буфер куба, а также один pipeline. Рендерятся в цикле: для каждого объекта — vkCmdBindDescriptorSets → vkCmdDrawIndexed. Это демонстрирует работу с несколькими наборами дескрипторов при общем пуле.

---

5. Результаты работы

В результате выполнения лабораторной работы получена программа, которая:

- Отображает куб в окне 1280×720 с использованием Vulkan;
- Позволяет переключаться между перспективной и ортографической проекцией;
- Даёт возможность изменять положение, поворот и масштаб каждого объекта через слайдеры;
- Анимирует движение кубов по окружности с настраиваемыми радиусом и скоростью;
- Позволяет менять цвет каждого объекта;
- Поддерживает процедурные цвета вершин;
- Отображает до 5 объектов одновременно, каждый со своим набором дескрипторов.

Скриншоты работы программы приведены в папке docs/screenshots репозитория.

---

6. Выводы

В ходе лабораторной работы были изучены и закреплены:

- Основы работы с Vulkan API: создание instance, устройства, swapchain, render pass, графического конвейера;
- Работа с буферами GPU через VMA: vertex, index, uniform-буферы, дескрипторные наборы;
- Матричные преобразования в 3D-графике: model-view-projection, разница между перспективной и ортографической проекциями;
- Интерактивное управление сценой через ImGui;
- Рендеринг нескольких объектов с использованием отдельных наборов дескрипторов;
- Анимация объектов по параметрическим траекториям.

Все дополнительные задания выполнены в полном объёме, что соответствует оценке «отлично».

---

7. Ссылка на репозиторий

https://github.com/sanechka26/CG_Lab_1